RTVRPTW · Documentación técnica

Modelo, algoritmo GRASP y guías de operación

Proyecto académico · Universidad de La Laguna

Noviembre de 2026

Índice

RTVRPTW · Documentación técnica **1**
- Objetivo . . . 2
- Componentes principales . . . 2
- Contenido de este portal . . . 2

Guía de uso **3**
- Preparar el proyecto . . . 3
- Compilar y ejecutar . . . 3
- Órdenes de Maven en terminal e IntelliJ . . . 3
- Documentación API . . . 3

Arquitectura del sistema **5**
- Vista general . . . 5
- Flujo de procesamiento . . . 5
- Relaciones relevantes . . . 5

Análisis de calidad con Maven y SonarQube **6**
- Herramientas y métricas . . . 6
- Cómo obtener resultados verificables . . . 6
- Problemas detectados al revisar el código original . . . 6
- Pruebas de regresión . . . 7
- Valoración crítica . . . 7

Registro breve de decisiones técnicas **8**
- D-01 · Mantener paquetes y heurística existentes . . . 8
- D-02 · Hacer explícitos los errores de entrada . . . 8
- D-03 · Construcción reproducible y pruebas mínimas . . . 8
- D-04 · Unificar documentación . . . 8
- D-05 · Demostrar deploy sin publicar en un servidor ajeno . . . 8

Apéndice · Referencia breve de clases Java **9**

> **Edición de consulta:** este PDF se construyó a partir de la documentación Markdown del proyecto. Para la entrega definitiva, se deberá sustituir por el PDF generado mediante **Doxygen + LaTeX**, según la actividad 9.4.

RTVRPTW · Documentación técnica

Recomendador Turístico con Ventanas de Tiempo — proyecto original ExpositoTOP, adaptado para la Práctica 3 de Laboratorio de Desarrollo y Herramientas (ULL).

Objetivo

El sistema selecciona puntos de interés turísticos que pueden visitarse mediante rutas sujetas a **ventanas de tiempo**, tiempos de servicio y disponibilidad de vehículos o rutas. La calidad de una solución se expresa mediante las puntuaciones acumuladas de los puntos visitados.

Componentes principales

- **Lector de instancias:** `TOPTWReader` transforma los ficheros de `Instances/TOPTW/` en objetos `TOPTW`.
- **Modelo y solución:** `TOPTW` conserva datos y distancias; `TOPTWSolution` representa las rutas.
- **Algoritmo:** `TOPTWGRASP` construye soluciones heurísticas mediante una lista restringida de candidatos (RCL).
- **Evaluación:** `TOPTWEvaluator` actualiza la puntuación obtenida.
- **Utilidades:** `BellmanFord`, `PowerSet`, `Pair` y `ExpositoUtilities`.

Contenido de este portal

La **guía de uso** explica compilación y ejecución; **arquitectura** expone las clases y sus relaciones; **calidad del código** prepara el análisis SonarQube; **decisiones técnicas** documenta las adaptaciones introducidas. La **referencia de clases** se genera automáticamente con MkDoxy a partir de los comentarios Java/Doxygen.

Este portal aplica *Docs-as-Code*: documentación Markdown versionada junto al programa y regenerada mediante comandos reproducibles.

Guía de uso

Preparar el proyecto

1. Instalar **JDK 17 o superior** y **Maven**.
2. Abrir la carpeta del proyecto en **IntelliJ IDEA** como proyecto Maven.
3. Confirmar que el IDE usa el JDK correcto y recargar las dependencias Maven.

Compilar y ejecutar

Desde la carpeta raíz del proyecto:

```
mvn clean verify
java -jar target/rtvrptw-1.0-SNAPSHOT.jar c101Recortada.txt 2 3
```

Los parámetros de la ejecución representan **archivo de instancia**, **número de iteraciones** y **tamaño máximo de la RCL**. El programa carga la instancia desde `Instances/TOPTW/`, construye una solución para cada iteración y muestra la media y la mejor puntuación. El comportamiento aleatorio implica que las puntuaciones no tienen que ser iguales en ejecuciones sucesivas.

Atención: sin argumentos se ejecuta el procesamiento original de 29 instancias, con tres valores de RCL y 10 000 iteraciones por configuración. No es adecuado para la primera prueba.

Órdenes de Maven en terminal e IntelliJ

Orden	Función	Panel Maven en IntelliJ
`mvn -version`	Comprobar instalación	Terminal integrada
`mvn validate`	Validar el modelo POM	Lifecycle → validate
`mvn compile`	Compilar las clases	Lifecycle → compile
`mvn test`	Ejecutar pruebas JUnit	Lifecycle → test
`mvn package`	Generar el archivo JAR	Lifecycle → package
`mvn install`	Instalar el JAR en `.m2`	Lifecycle → install
`mvn deploy`	Publicar en el repositorio local de demostración	Lifecycle → deploy
`mvn clean`	Limpiar `target/`	Lifecycle → clean
`mvn verify`	Ejecutar comprobaciones y emitir informe JaCoCo	Lifecycle → verify

Fases encadenadas: package incluye las fases anteriores del ciclo *default*; `install` incluye también package; `deploy` incluye las anteriores. `clean` pertenece a un ciclo independiente.

Documentación API

Instalar Doxygen, Graphviz y los paquetes de Python indicados en `requirements-docs.txt`. Ejecutar:

```
doxygen Doxyfile
mkdocs build --strict
mkdocs serve
```

El servidor de vista previa suele mostrarse en `http://127.0.0.1:8000/`; el HTML final queda en `site/`. El PDF nativo de la referencia se compila desde `target/doxygen/latex/` con `make` o `make.bat` y LaTeX disponible.

Arquitectura del sistema

Vista general

El código se organiza en dos paquetes principales y un paquete de tipos auxiliares:

- **`top`**: núcleo TOPTW, algoritmo, lector, solución y punto de entrada.
- **`es.ull.esit.utilities`**: funciones auxiliares de grafos, matemáticas y generación de subconjuntos.
- **`es.ull.esit.utils`**: clase genérica `Pair`.

Flujo de procesamiento

1. La clase `mainTOPTW` selecciona la instancia y el número de iteraciones.
2. `TOPTWReader.readProblem()` lee los datos del archivo y genera un `TOPTW` con matriz de distancias.
3. `TOPTWSolution` crea una estructura para registrar rutas, sucesores y predecesores.
4. `TOPTWGRASP` construye soluciones con una lista restringida de candidatos que cumplen las restricciones evaluadas por el algoritmo.
5. `TOPTWSolution.evaluateFitness()` suma la puntuación de los puntos visitados; `TOPTWEvaluator` puede almacenar ese valor como función objetivo.
6. El programa muestra la solución y un resumen de las puntuaciones de las iteraciones.

Relaciones relevantes

Clase	Responsabilidad	Colaboración
`TOPTW`	Mantener coordenadas, ventanas temporales, puntuaciones y distancias	`TOPTWReader`, `TOPTWSolution`
`TOPTWReader`	Analizar el formato de entrada	`TOPTW`, `ExpositoUtilities`
`TOPTWSolution`	Mantener rutas y evaluación de su puntuación	`TOPTW`, `TOPTWGRASP`
`TOPTWGRASP`	Heurística aleatorizada	`TOPTWSolution`
`TOPTWEvaluator`	Guardar la evaluación de la solución	`TOPTWSolution`
`BellmanFord`, `PowerSet`, `Pair`	Apoyo reutilizable	Código de utilidad

Los **diagramas estructurales** se generan en la referencia Doxygen con Graphviz (`HAVE_DOT = YES`). No se añaden patrones ni capas nuevas porque la práctica se centra en calidad, compilación y documentación del proyecto existente.

Análisis de calidad con Maven y SonarQube

Herramientas y métricas

Maven construye el JAR y ejecuta las pruebas JUnit. JaCoCo genera `target/site/jacoco/jacoco.xml`. SonarQube recibe los resultados al ejecutar SonarScanner for Maven e informa, entre otras dimensiones, de **fiabilidad**, **seguridad**, **mantenibilidad**, **duplicaciones** y **cobertura**.

Cómo obtener resultados verificables

1. Ejecutar `mvn clean verify` y comprobar que las pruebas pasan.
2. Levantar SonarQube, crear el proyecto y configurar el token (`SONAR_TOKEN`).
3. Ejecutar el comando documentado en el README.
4. Abrir el panel de SonarQube y registrar los valores reales de incidencias por severidad o tipo, cobertura, duplicación y estado de *Quality Gate*.
5. Seleccionar 2-3 incidencias representativas y documentar **archivo, línea, regla y propuesta**. Adjuntar capturas antes/después si se repite el análisis.

No se atribuyen cifras a SonarQube sin haber ejecutado el servidor: el análisis estático es independiente de la revisión manual del código.

Problemas detectados al revisar el código original

Problema verificado en el original	Riesgo	Corrección aplicada
`TOPTWReader.readProblem()` llamaba a `System.exit(0)` cuando había un `IOException`	Termina toda la aplicación y oculta el fallo como salida con éxito	Propagación mediante `UncheckedIOException` y cierre automático del lector
`ExpositoUtilities.printFile()` podía invocar `reader.close()` cuando `reader` era `null`	`NullPointerException` en el bloque `finally`	`try-with-resources` y registro del error
`TOPTWEvaluator.evaluate()` no efectuaba evaluación (código comentado)	Valor objetivo sin actualizar	Asignación de `solution.evaluateFitness()`
`PowerSet.next()` utilizaba `TreeSet` con elementos genéricos	`ClassCastException` para elementos no comparables	`LinkedHashSet` y excepción estándar al agotarse
`GRASP()` dividía entre cero si se pedían cero iteraciones y la selección fallaba con RCL vacía	Operaciones inválidas y errores de ejecución	Validación de parámetros y excepción explícita

Estas son **observaciones por inspección del código original**, no hallazgos atribuidos automáticamente a SonarQube. Quedan aspectos mejorables como métodos largos, convenciones de nombres y lógica de heurística que se han conservado para no alterar innecesariamente el comportamiento del proyecto académico.

Pruebas de regresión

Las pruebas de `src/test/java/` cubren carga de datos, distancias simétricas, fallo de archivo inexistente, evaluación de una solución vacía, validación de parámetros y combinaciones de subconjuntos. Son pruebas pequeñas y enfocadas; **no equivalen a una validación completa de la heurística**.

Valoración crítica

Maven mejora la reproducibilidad y simplifica la construcción, pero no garantiza por sí solo ausencia de defectos. SonarQube señala oportunidades de mejora; cada hallazgo debe interpretarse conforme al contexto. Doxygen reduce el coste de mantener la referencia técnica, pero depende de comentarios comprensibles y actualizados. La cobertura cuantifica ejecución de código, no corrección lógica.

Registro breve de decisiones técnicas

Este registro resume las decisiones de adaptación para la práctica, sin introducir nuevas funcionalidades ajenas al proyecto original.

D-01 · Mantener paquetes y heurística existentes

Decisión: trasladar las clases existentes a `src/main/java` sin rediseñar el algoritmo GRASP. **Motivo:** el objetivo de la actividad es evaluar herramientas de calidad y documentación, no reimplementar la lógica del recomendador. **Consecuencia:** persisten algunas oportunidades de refactorización que deben analizarse con SonarQube.

D-02 · Hacer explícitos los errores de entrada

Decisión: reemplazar `System.exit(0)` en el lector por `UncheckedIOException` y cerrar recursos automáticamente. **Motivo:** permitir que IDE, pruebas y ejecución por lotes distingan un archivo inválido de una ejecución correcta.

D-03 · Construcción reproducible y pruebas mínimas

Decisión: declarar Java 17, JUnit 5 y plugins Maven con versión fija; habilitar JaCoCo para que SonarQube pueda importar cobertura. **Motivo:** facilitar ejecuciones equivalentes por terminal e IntelliJ.

D-04 · Unificar documentación

Decisión: usar Doxygen para comentarios y diagramas, y MkDocs Material con MkDoxy para publicar manuales Markdown y referencia de código. **Motivo:** cumplir el enfoque Docs-as-Code. Los textos explicativos se han elaborado con asistencia de un LLM y revisado a partir del código original; **no** sustituyen la medición real de SonarQube.

D-05 · Demostrar deploy sin publicar en un servidor ajeno

Decisión: configurar en Maven un repositorio de archivos local en `target/repositorio-maven/`. **Motivo:** probar la fase de despliegue con un destino verificable sin requerir credenciales externas. **Límite:** no es un despliegue de la aplicación a producción ni equivale a su publicación en GitHub.

Apéndice · Referencia breve de clases Java

La referencia completa, con métodos, miembros y diagramas Graphviz, se produce automáticamente con doxygen `Doxyfile` y se integra en MkDocs mediante MkDoxy.

Clase	Paquete	Responsabilidad
`TOPTW`	`top`	Datos de entrada y distancias
`TOPTWReader`	`top`	Lectura de instancias
`TOPTWSolution`	`top`	Representación y evaluación de rutas
`TOPTWGRASP`	`top`	Heurística GRASP
`TOPTWEvaluator`	`top`	Almacenar función objetivo
`TOPTWRoute`	`top`	Enlaces de una ruta
`mainTOPTW`	`top`	Punto de entrada
`BellmanFord`	`es.ull.esit.utilities`	Funciones de grafo
`PowerSet`	`es.ull.esit.utilities`	Generar subconjuntos
`ExpositoUtilities`	`es.ull.esit.utilities`	Formateo, archivos y matrices
`Pair`	`es.ull.esit.utils`	Par genérico